

11 - Explicacao: Por que vetor precisa estar ordenado para busca binaria?

Resposta teorica:

A busca binaria funciona dividindo o vetor ao meio e comparando o elemento do meio com o alvo. Para decidir se vai buscar na metade ESQUERDA ou DIREITA, ela PRECISA saber se os elementos estao em ordem.

Exemplo pratico com vetor NAO ordenado:

Vetor: [5, 2, 8, 1, 9]

Alvo: 1

Passo 1: meio = posicao 2 (valor 8)

Comparacao: $8 == 1$? Nao

$8 > 1$? Vou para esquerda? Mas os valores a esquerda sao [5,2] - o 1 esta a DIREITA!

Conclusao: Nao da para saber se o alvo esta a esquerda ou direita porque os valores nao estao em ordem!

Com vetor ORDENADO:

Vetor: [1, 2, 5, 8, 9]

Alvo: 1

Passo 1: meio = posicao 2 (valor 5)

Comparacao: $5 > 1$? SIM! Entao o 1 so pode estar a ESQUERDA

Busca em [1,2] e encontra o 1.

Conclusao: A ordenacao permite descartar metade do vetor a cada passo!

13 - Determinacao da Complexidade

a)

```
for (i = 0; i < n; i++) {  
    printf("%d", i);  
}
```

Complexidade: $O(n)$ - Linear

Explicacao: O loop executa exatamente n vezes. Cada iteracao faz uma operacao constante (printf). Se $n = 1000$, executa 1000 vezes. Se n dobra, o tempo dobra.

b)

```
for (i = 0; i < n; i++) {  
    for (j = 0; j < n; j++) {  
        printf("%d", i+j);  
    }  
}
```

Complexidade: $O(n^2)$ - Quadratica

Explicacao: Loop externo executa n vezes. Para CADA iteracao do externo, o interno executa n vezes. Total = $n * n = n^2$ operacoes. Se $n = 10 \rightarrow 100$ operacoes. Se $n = 100 \rightarrow 10.000$ operacoes.

c)

```
for (i = 1; i < n; i *= 2) {  
    printf("%d", i);  
}
```

Complexidade: $O(\log n)$ - Logaritmica

Explicacao: A variavel i dobra a cada iteracao: 1, 2, 4, 8, 16, 32...
Numero de iteracoes = $\log_2(n)$. Se $n = 1024 \rightarrow$ apenas 10 iteracoes. Se n dobra, aumenta apenas 1 iteracao.

16 - Classificacao (Melhor, Pior e Medio Caso)

Busca Linear

Caso	Complexidade	Explicacao
Melhor caso	$O(1)$	Elemento esta na primeira posicao (encontra em 1 comparacao)
Pior caso	$O(n)$	Elemento esta na ultima posicao OU nao existe (percorre tudo)
Medio caso	$O(n)$	Em media, precisa percorrer metade do vetor ($n/2$)

Busca Binaria

Caso	Complexidade	Explicacao
Melhor caso	$O(1)$	Elemento esta exatamente no meio (encontra na 1a tentativa)
Pior caso	$O(\log n)$	Elemento so e encontrado na ultima divisao OU nao existe
Medio caso	$O(\log n)$	Mesmo comportamento do pior caso, pois descarta metade sempre

Tabela Resumo:

Algoritmo	Melhor	Pior	Medio
Busca Linear	$O(1)$	$O(n)$	$O(n)$
Busca Binaria	$O(1)$	$O(\log n)$	$O(\log n)$

19 - Comparacao de Algoritmos de Ordenacao

Insertion Sort

Caso	Complexidade	Explicacao
Melhor caso	$O(n)$	Vetor ja ordenado (apenas compara, nao desloca)
Pior caso	$O(n^2)$	Vetor em ordem reversa (desloca muitos elementos)
Medio caso	$O(n^2)$	Em media, desloca cerca de $n/2$ elementos por vez

Funcionamento: Como ordenar cartas de baralho - pega um elemento e insere na posicao correta.

Tabela Comparativa Completa:

Algoritmo	Melhor Caso	Pior Caso	Medio Caso	Estavel?	In-place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sim	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Nao	Sim
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sim	Sim

Bubble Sort

Caso	Complexidade	Explicacao
Melhor caso	$O(n)$	Vetor ja ordenado (com a otimizacao, faz apenas 1 passagem)
Pior caso	$O(n^2)$	Vetor em ordem reversa (faz todas as comparacoes e trocas)
Medio caso	$O(n^2)$	Em media, precisa de $n^2/2$ comparacoes

Funcionamento: Compara pares adjacentes e troca se estiverem na ordem errada. Os maiores elementos "flutuam" para o final.

Qual e o melhor?

Cenario	Melhor escolha	Porque
Vetor pequeno ($n < 50$)	Insertion Sort	Simples e eficiente
Vetor quase ordenado	Insertion Sort	Melhor caso $O(n)$
Vetor grande ($n > 1000$)	Nenhum dos 3	Precisa de algoritmo mais rapido (Quick Sort, Merge Sort)
Memoria muito limitada	Selection Sort	Faz menos trocas que os outros

Exemplo pratico com $n = 1000$ elementos:

Algoritmo	Melhor caso	Pior caso	Medio caso
Bubble Sort	1000 operacoes	1.000.000	500.000
Selection Sort	1.000.000	1.000.000	1.000.000
Insertion Sort	1000 operacoes	1.000.000	250.000

Conclusao: Insertion Sort e o melhor entre os tres para a maioria dos casos praticos com dados pequenos ou quase ordenados.